بخش دوم: الگوریتم‌های زمان‌بندی (Algorithms)

در این بخش به بررسی استراتژی‌های مختلف برای انتخاب پردازه بعدی می‌پردازیم. فرض ما بر این است که فقط یک هسته پردازشی داریم.

۱. الگوریتم اولین ورود، اولین سرویس (FCFS)

نام کامل: First-Come, First-Served **ساختار:** صف ساده (FIFO Queue).

- **روش کار:** پردازه‌ای که زودتر درخواست دهد، زودتر CPU را می‌گیرد. وقتی CPU به آن داده شد، تا پایان کارش آن را رها نمی‌کند.
- **نوع: غیرپس‌گرفتنی** (Non-preemptive).

مثال و محاسبه (بسیار مهم)

فرض کنید سه پردازه با زمان‌های انفجار (Burst Time) زیر در لحظه صفر وارد شوند:

- پردازه 1P: برابر ۲۴ میلی‌ثانیه
- پردازه 2P: برابر ۳ میلی‌ثانیه
- پردازه 3P: برابر ۳ میلی‌ثانیه

حالت اول: ترتیب ورود 3P1, P2, P

1. 1P از ۰ تا ۲۴ اجرا می‌شود. (انتظار: ۰)
2. 2P از ۲۴ تا ۲۷ اجرا می‌شود. (انتظار: ۲۴)
3. 3P از ۲۷ تا ۳۰ اجرا می‌شود. (انتظار: ۲۷)

میانگین زمان انتظار: ۱۷ میلی‌ثانیه = ۳ / (۲۷ + ۲۴ + ۰)

حالت دوم: ترتیب ورود 1P2, P3, P (اگر پردازه‌های کوتاه اول بیایند)

- میانگین زمان انتظار: ۳ میلی‌ثانیه = ۳ / (۳ + ۰ + ۶)
- **نتیجه:** ترتیب ورود در FCFS تاثیر وحشتناکی روی کارایی دارد.

اثر بدرقه (Convoy Effect)

زمانی رخ می‌دهد که یک پردازه **بسیار طولانی** (CPU-Bound) در جلوی صف باشد و تعداد زیادی پردازه **کوتاه** (I/O-Bound) پشت سر آن گیر کنند.

نتیجه: کاهش شدید بهره‌وری دستگاه‌های I/O و پردازنده. (همه منتظر آن غول هستند تا تمام شود).

۲. الگوریتم کوتاه‌ترین کار، اول (SJF)

نام کامل: Shortest-Job-First (یا دقیق‌تر: Shortest-Next-CPU-Burst)

- **روش کار:** پردازه‌ای انتخاب می‌شود که **کوتاه‌ترین انفجار بعدی CPU** را داشته باشد.
- **ویژگی کلیدی:** این الگوریتم **بهینه (Optimal)** است؛ یعنی کمترین میانگین زمان انتظار را در بین تمام الگوریتم‌ها تضمین می‌کند.

چالش پیش‌بینی (Prediction)

مشکل SJF این است که سیستم‌عامل نمی‌داند انفجار بعدی پردازه چقدر طول می‌کشد. راه حل، **پیش‌بینی** بر اساس سوابق قبلی است.

فرمول میانگین‌گیری نمایی: (Next_Prediction = (alpha * Last_Actual) + ((1 - alpha) * (Last_Prediction

- **alpha (آلفا):** ضریب وزنی (بین ۰ و ۱).
- اگر alpha = 0: تاریخچه قبلی مهم است (تغییرات جدید نادیده گرفته می‌شود).
- اگر alpha = 1: فقط مقدار واقعی آخر مهم است (حافظه کوتاه مدت).
- معمولاً alpha = 0.5 در نظر گرفته می‌شود.

انواع SJF

۱. غیرپس‌گرفتنی (Non-preemptive): وقتی CPU به پردازه داده شد، حتی اگر پردازه جدیدی با کار کمتر بیاید، قطع نمی‌شود.

۲. پس‌گرفتنی (Preemptive) یا SRTF: نام دیگر: **Shortest-Remaining-Time-First** (کوتاه‌ترین زمان باقی‌مانده، اول).

روش: اگر پردازه‌ای جدید وارد شود که طول انفجارش از **زمان باقی‌مانده** پردازه فعلی کمتر باشد، پردازه فعلی قطع (Preempt) می‌شود.

مثال SRTF (تست‌خیزترین بخش)

پردازه	زمان	زمان

	ورود	انفجار
1P	0	8
2P	1	4
3P	2	9
4P	3	5

تحلیل گانت چارت:

1. **زمان ۰:** 1P شروع می‌کند.
2. **زمان ۱:** 2P می‌رسد (طول ۴). 1P باقی‌مانده‌اش ۷ است. چون (۴ کوچکتر از ۷) است، پس 1P قطع می‌شود و 2P می‌آید.
3. **زمان ۵:** 2P تمام می‌شود. 4P (طول ۵) از 1P (باقی‌مانده ۷) و 3P (طول ۹) کمتر است. پس 4P می‌آید.
4. **زمان ۱۰:** 4P تمام می‌شود. 1P (باقی‌مانده ۷) از 3P (طول ۹) کمتر است. پس 1P می‌آید.
5. **زمان ۱۷:** 1P تمام می‌شود. 3P می‌آید.

میانگین زمان انتظار: ۶.۵ میلی‌ثانیه. (در حالت غیرپس‌گرفتنی ۷.۷۵ می‌شد).

۳. الگوریتم نوبت چرخشی (Round Robin - RR)

این الگوریتم مخصوص سیستم‌های **اشتراک زمانی** (Time-Sharing) است.

- **روش کار:** شبیه FCFS است، اما **پس‌گرفتنی**.
- **کوانتوم زمانی (Time Quantum):** یک بازه زمانی کوچک (مثلاً ۱۰ تا ۱۰۰ میلی‌ثانیه) تعریف می‌شود.

قانون اجرا: هر پردازه حداکثر به اندازه ۱ کوانتوم CPU را می‌گیرد.

1. اگر کارش زودتر تمام شد -> CPU را رها می‌کند (Voluntary).
2. اگر کارش تمام نشد -> تایمر وقفه می‌دهد، پردازه قطع می‌شود و به **ته صف آماده** می‌رود.

تأثیر اندازه کوانتوم (q)

- **اگر q بسیار بزرگ باشد:** الگوریتم تبدیل به FCFS می‌شود.
- **اگر q بسیار کوچک باشد:** سربار سیستم زیاد می‌شود. (مثلاً اگر کوانتوم ۱ میلی‌ثانیه باشد و تعویض متن هم ۱ میلی‌ثانیه طول بکشد، ۵۰٪ توان CPU هدر می‌رود!).

- **قانون سرانگشتی:** کوانتوم باید اندکی بزرگتر از زمان مورد نیاز برای ۸۰٪ انفجارهای CPU باشد تا تعویض متن به حداقل برسد.

۴. زمان‌بندی اولویتی (Priority Scheduling)

- **روش کار:** به هر پردازه یک عدد (اولویت) نسبت داده می‌شود. CPU به پردازه با **بالاترین اولویت** داده می‌شود.
- **نکته:** در برخی سیستم‌ها عدد کمتر به معنی اولویت بالاتر است و برعکس.
- **نکته:** الگوریتم SJF در واقع یک نوع اولویت‌دهی است که در آن اولویت برابر با عکسِ زمان انفجار بعدی است (زمان کمتر = اولویت بیشتر).

مشکل بزرگ: گرسنگی (Starvation) - که در بخش بعدی توضیح داده شد.

پایان بخش دوم از فصل ۵